Pipeline

Пайплайн (pipeline) в контексте машинного обучения — это последовательность шагов, объединяющая различные этапы обработки данных и обучения модели. Он позволяет автоматизировать и упрощать процесс разработки моделей, обеспечивая структурированное выполнение всех необходимых операций. Основные компоненты пайплайна:

– извлечение данных: Сбор данных из различных источников, таких как базы данных или API. Это первый шаг, который включает в себя преобразование данных в нужный формат для дальнейшей обработки.

– предобработка данных: Очищение данных (скраббинг), включая заполнение пропусков, удаление дубликатов и исправление ошибок. Этот этап критически важен, так как качество данных напрямую влияет на производительность модели

– разведочный анализ данных (EDA): Анализ и визуализация данных для выявления закономерностей и аномалий. Это помогает лучше понять данные и определить, какие признаки могут быть полезны для модели

– моделирование: Обучение модели на подготовленных данных. Этот этап включает выбор алгоритма и его настройку (гиперпараметры) для достижения наилучших результатов

– оценка модели: Использование тестовых данных для оценки производительности модели с помощью метрик, таких как точность, полнота и F1-мера

– интерпретация результатов: Анализ полученных результатов и их представление заинтересованным сторонам. Важно уметь объяснять, как модель принимает решения и какие выводы можно сделать на основе ее предсказаний

– пересмотр и обновление: Регулярное обновление модели по мере изменения бизнес-условий или появления новых данных. Это позволяет поддерживать актуальность и точность модели

Преимущества использования пайплайнов

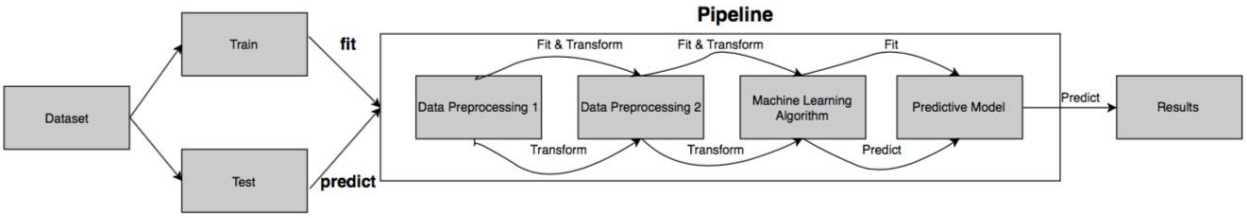
Автоматизация: пайплайн позволяет автоматизировать весь процесс от извлечения данных до предсказания, что снижает вероятность ошибок и экономит время.

Повторяемость: структурированный подход делает процесс разработки моделей более предсказуемым и повторяемым.

Упрощение тестирования: легче тестировать различные компоненты пайплайна по отдельности и в комбинации.

Лучшая организация кода: позволяет организовать код более логично и понятно, что облегчает его поддержку и развитие.

Схематически работу пайплайна можно представить следующим образом:



Пример реализации В библиотеке Scikit-learn создание пайплайна может выглядеть следующим образом:

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# Создание пайплайна
pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),  # Заполнение пропусков
    ('scaler', StandardScaler()),                  # Масштабирование
признаков
    ('classifier', LogisticRegression())           # Обучение классификатора
])

# Обучение пайплайна на тренировочных данных
pipeline.fit(X_train, y_train)

# Предсказание на тестовых данных
predictions = pipeline.predict(X_test)
```

В этом примере пайплайн включает три этапа: заполнение пропусков, масштабирование признаков и обучение логистической регрессии. Такой подход позволяет легко управлять процессом обработки данных и обучения модели, делая его более эффективным и понятным.

Разберем пример связи метрик DS и бизнес метрик

Владелец новостного портала зарабатывает 1р. на показе рекламы каждому пользователю, который пришел на его сайт и получает штраф 3р. за каждый опубликованный токсичный комментарий на его сайте. Необходимо блокировать пользователей, которые оставляют токсичные комментарии. Задача состоит в том, чтобы классифицировать токсичные/не токсичные комментарии таким образом, чтобы доход владельца ресурса был максимальным.

Имеем такой набор данных:

	id	comment_text	toxic	severe_toxic	obscene	threat	insult	identity_hate
129812	b66fb7d9a7a02eee	"\n\nNancy Reagan\nThanks for asking. I remov...	0	0	0	0	0	0
11866	1f6c7e193783b932	Thanks for your help on Turtle Racing. It woul...	0	0	0	0	0	0
42118	7060848b4b6cc9d7	"\nOh, come on , that is a weak effort. Make i...	0	0	0	0	0	0

Так есть поле «comment_text», содержащее сам комментарий и поле «toxic», которое является меткой токсичности. Т.е. имеем задачу бинарной классификации. Проверим соотношение классов:

```
train['y'] = train[class_names].max(axis=1).values
train['y'].value_counts()
```

y	count
0	143346
1	16225

dtype: int64

Разделим данные на train/test

```
X_train, X_test, y_train, y_test = train_test_split(train.drop(columns='y'),
                                                    train['y'], random_state=0)
```

Построим простое векторное представление текстов и обучим логистическую регрессию

```
%%time
# соберем наш простой pipeline, но нам понадобится написать класс для выбора
нужного поля
class FeatureSelector(BaseEstimator, TransformerMixin):
    def __init__(self, column):
        self.column = column

    def fit(self, X, y=None):
        return self

    def transform(self, X, y=None):
        return X[self.column]

classifier = Pipeline([
('comment_text_selector', FeatureSelector(column='comment_text')),
 ('comment_text_tfidf', TfidfVectorizer(strip_accents='unicode',
                                        analyzer='word',
                                        token_pattern=r'\w{1,}',
                                        stop_words='english',
                                        max_features=10000)),
                       ('clf', LogisticRegression())])

# запустим кросс-валидацию
cv_scores = cross_val_score(classifier, X_train, y_train, cv=3,
scoring='roc_auc')
cv_score = np.mean(cv_scores)
print(f'CV score is {cv_score}')
```

```
# обучим пайплайн на всем тренировочном датасете
classifier.fit(X_train, y_train)
y_score = classifier.predict_proba(X_test)[:, 1]
```

Тут создается объект `Pipeline`, который позволяет объединить несколько шагов обработки данных и обучения модели в один непрерывный процесс. `FeatureSelector`, который, извлекает текст комментариев из набора данных. Он выбирает столбец с именем `'comment_text'`.

Здесь `TfidfVectorizer` преобразует текст в числовые векторы:

– `strip_accents='unicode'`: убирает акценты из букв (например, `é` станет `e`, `ñ` станет `n`, `ü` станет `u`).

– `analyzer='word'`: анализирует текст на уровне слов.

– `token_pattern=r'\w{1,}'`: определяет, какие токены считать (слова длиной от 1 символа).

– `stop_words='english'`: игнорирует распространенные английские слова (стоп-слова).

– `max_features=10000`: ограничивает количество признаков до 10,000 наиболее значимых.

В качестве классификатора используется LogisticRegression. Эта модель будет обучаться на векторизованных данных, чтобы предсказывать классы на основе текста комментариев.

Метрики после кросс валидации получились следующими: 0.96450773, 0.96529758, 0.96691709, что говорит о хорошем качестве модели. Используя специализированные инструменты sklearn получим оптимальный порог отсечки:

```
import numpy as np
from sklearn.metrics import precision_recall_curve, roc_curve, roc_auc_score,
confusion_matrix

b = 1

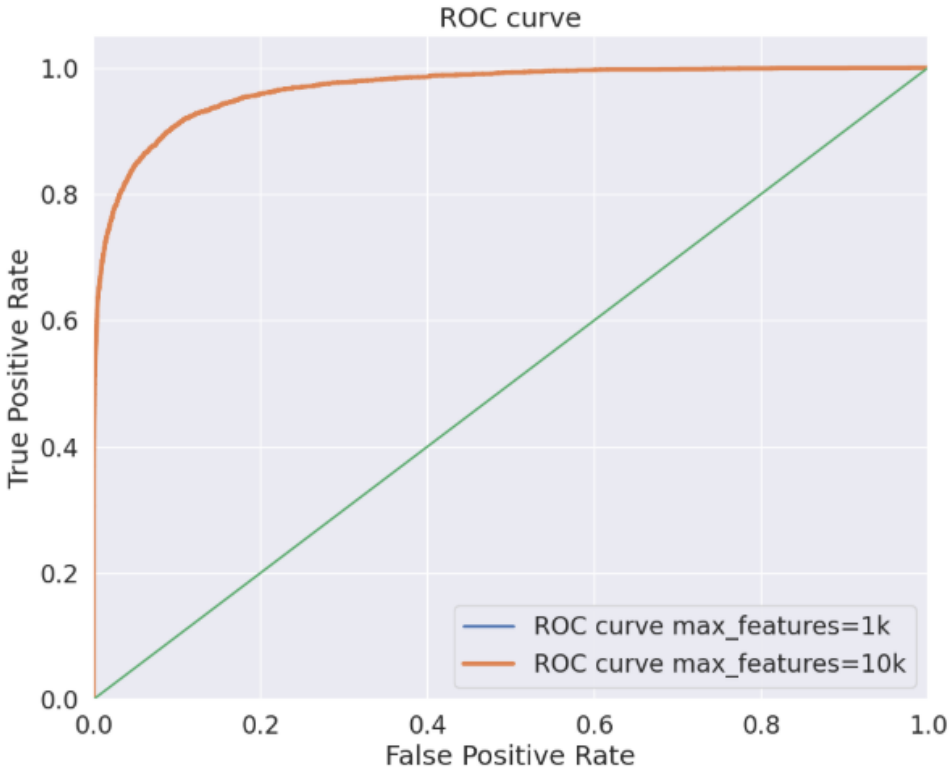
precision, recall, thresholds = precision_recall_curve(y_test.values,
y_score)
fscore = (1 + b**2) * (precision * recall) / (b**2 * precision + recall)
# locate the index of the largest f score
ix = np.argmax(fscore)
print(f'Best Threshold={thresholds[ix]}, F-Score={fscore[ix]:.3f},
Precision={precision[ix]:.3f}, Recall={recall[ix]:.3f}')
```

Получаем:

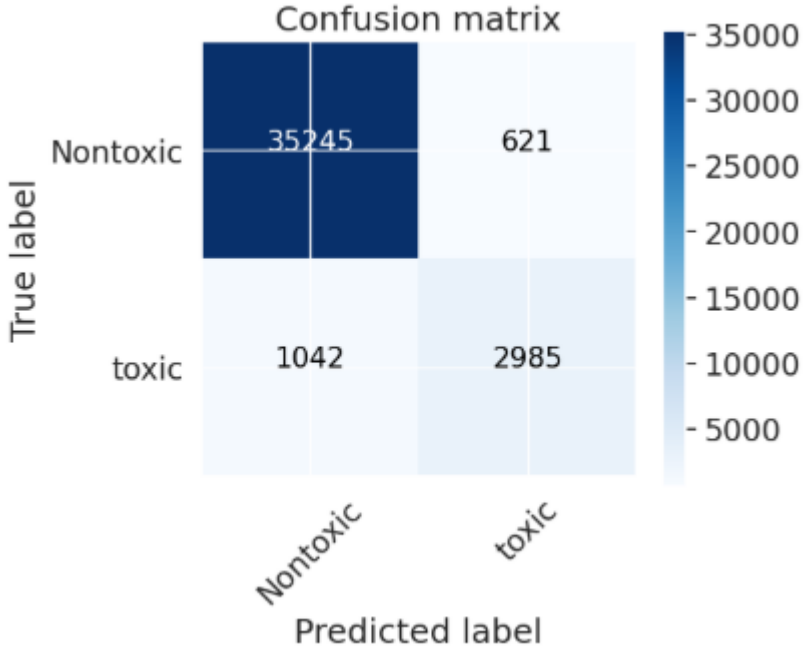
```
Best Threshold=0.285432324003, F-Score=0.782, Precision=0.828, Recall=0.741
```

Получается, что оптимальный порог отсечки с точки зрения мтерик машинного обучения равен 0,285. При таком пороге самое высокое значение

метрики F-Score и оно действительно хорошее. При таком значении F-Score Precision составляет 0.828, т.е. классификатор хорошо отделяет целевой класс от нецелевого и Recall=0.741 говорит о том, что выбрано 74% объектов целевого класса (токсичных комментариев). Кривая ROC AUC говорит о том, что классификатор хорошо справляется со своей задачей



Рассмотрим матрицу ошибок

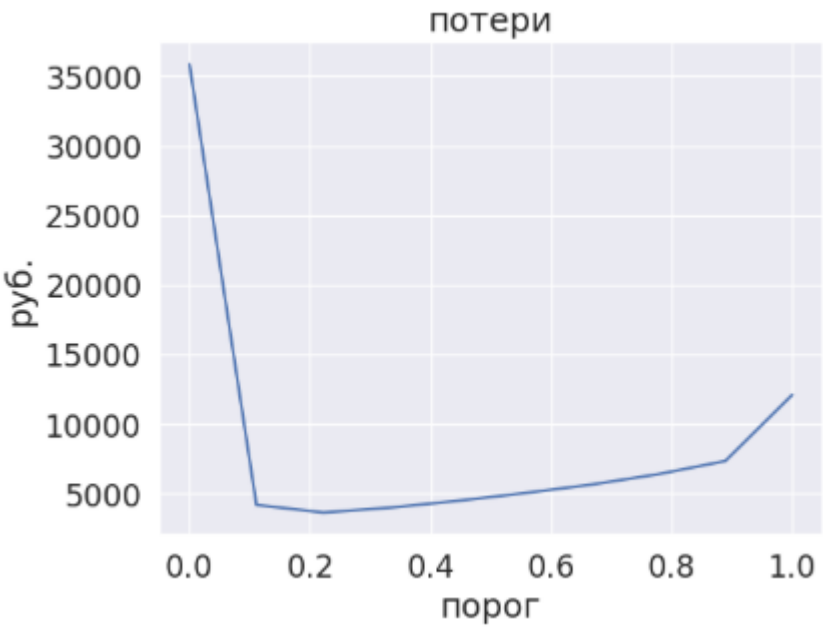


Видно, что для порогового значения отсечки равной 0,285 было заблокировано 621 человек, чьи комментарии система посчитала ошибочно токсичными, т.е. владелец сайта потерял 621р (каждый человека просмотревший рекламу приносит владельцу сайта 1р прибыли) и 1042 человека опубликовали свои токсичные комментарии, т.к. система их ошибочно приняла за хорошие, т.е. владелец ресурса получит штраф 1042*3=3126р. Всего убытки составят 3747р. Каким образом можно уменьшить финансовые потери владельца сайта? Можем поэкспериментировать с порогом отсечки. Сгенерируем 10 пороговых значений для отсечки и для каждого из них посчитаем матрицу потерь и финансовые потери владельца сайта. Так нам значение False Negative необходимо будет умножить на 3 и прибавить значение False Positive. Построим график потерь для 10 порогов.

```
pay_toxic_th = []
trh_level = np.linspace(0, 1, 10).tolist()

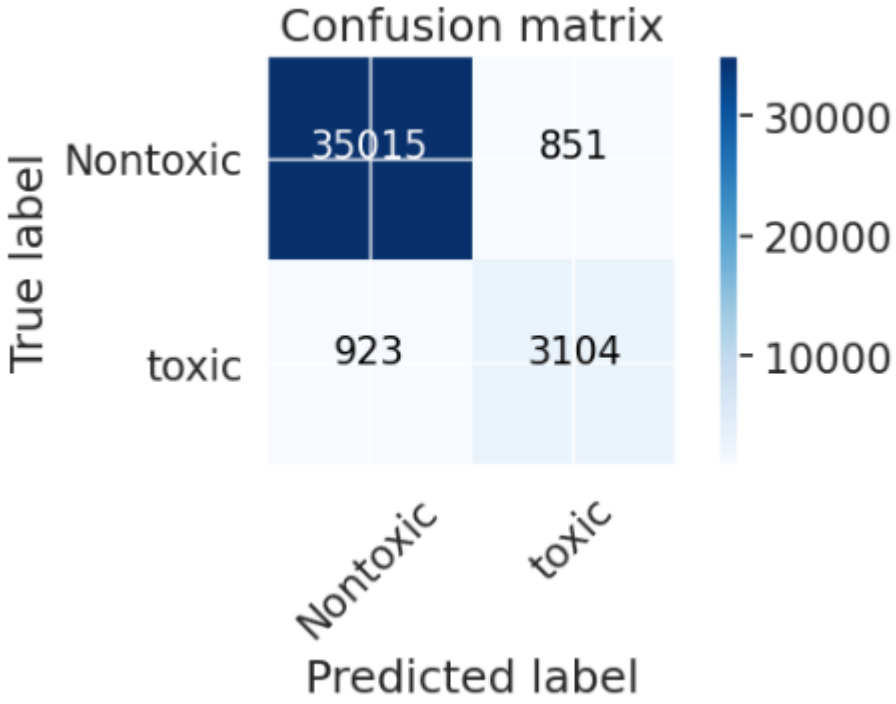
for trh in trh_level:
    cnf_matrix = confusion_matrix(y_test, y_score > trh)
    FN = cnf_matrix[1][0]
    FP = cnf_matrix[0][1]
    pay_toxic_th.append(FN * 3 + FP * 1)

plt.plot(trh_level, pay_toxic_th)
plt.ylabel('руб.')
plt.xlabel('порог')
plt.title('потери')
plt.show()
```



По графику видно, но минимум потерь достигается не при пороге отсечки 0,285 (что было наилучшим по метрикам DS), а 0,22. Тогда применив

неоптимальный с точки зрения метрик машинного обучения новый порог получим матрицу ошибок:



Видно, что увеличилось количество людей, которые не просмотрят рекламу, и уменьшилось количество пропущенных токсичных комментариев. Убытки составят 3620р., что на 127р. меньше, чем при оптимальном пороге с точки зрения метрик машинного обучения. По разобранному примеру можно сделать вывод, что оптимальные метрики машинного обучения могут быть не оптимальными для бизнеса.